

## 输入输出流

教 师：冀荣华

单 位：人工智能系

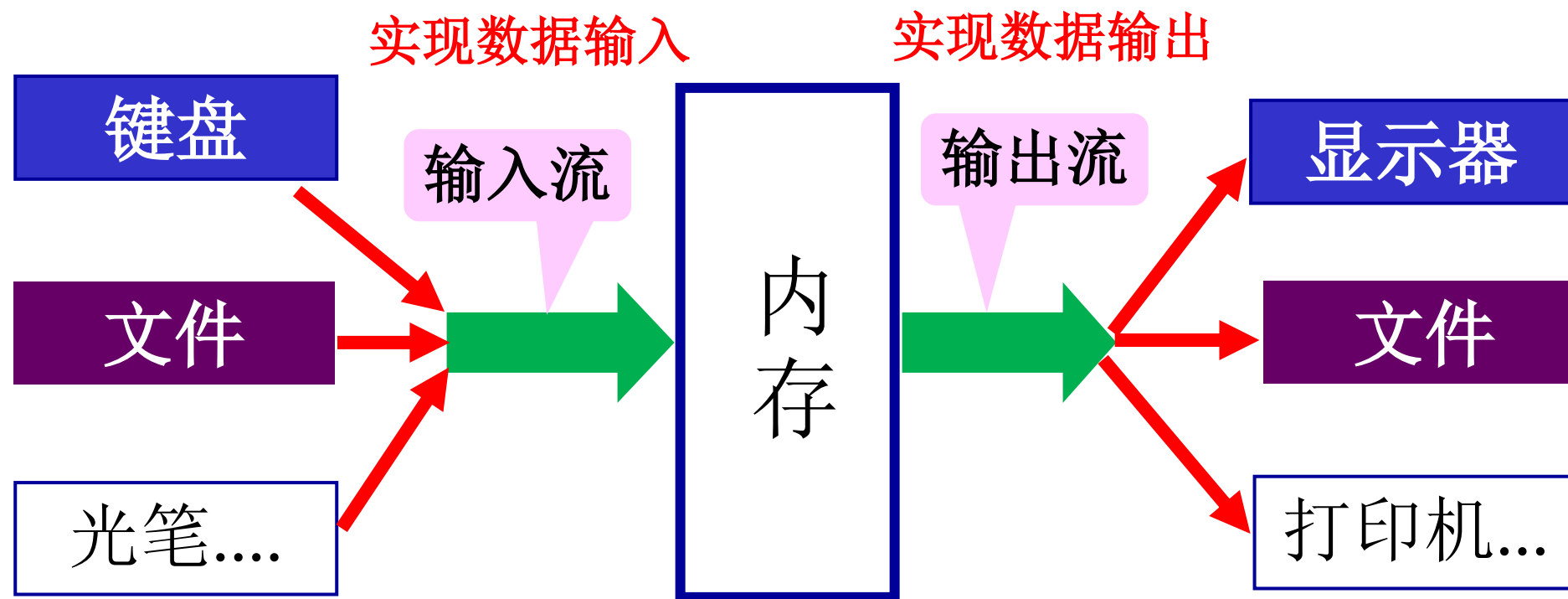


## 主要内容

- 一、C++输入输出
- 二、标准输出流
- 三、标准输入流
- 四、对数据文件的操作与文件流
- 五、字符串流



# 一、C++输入输出

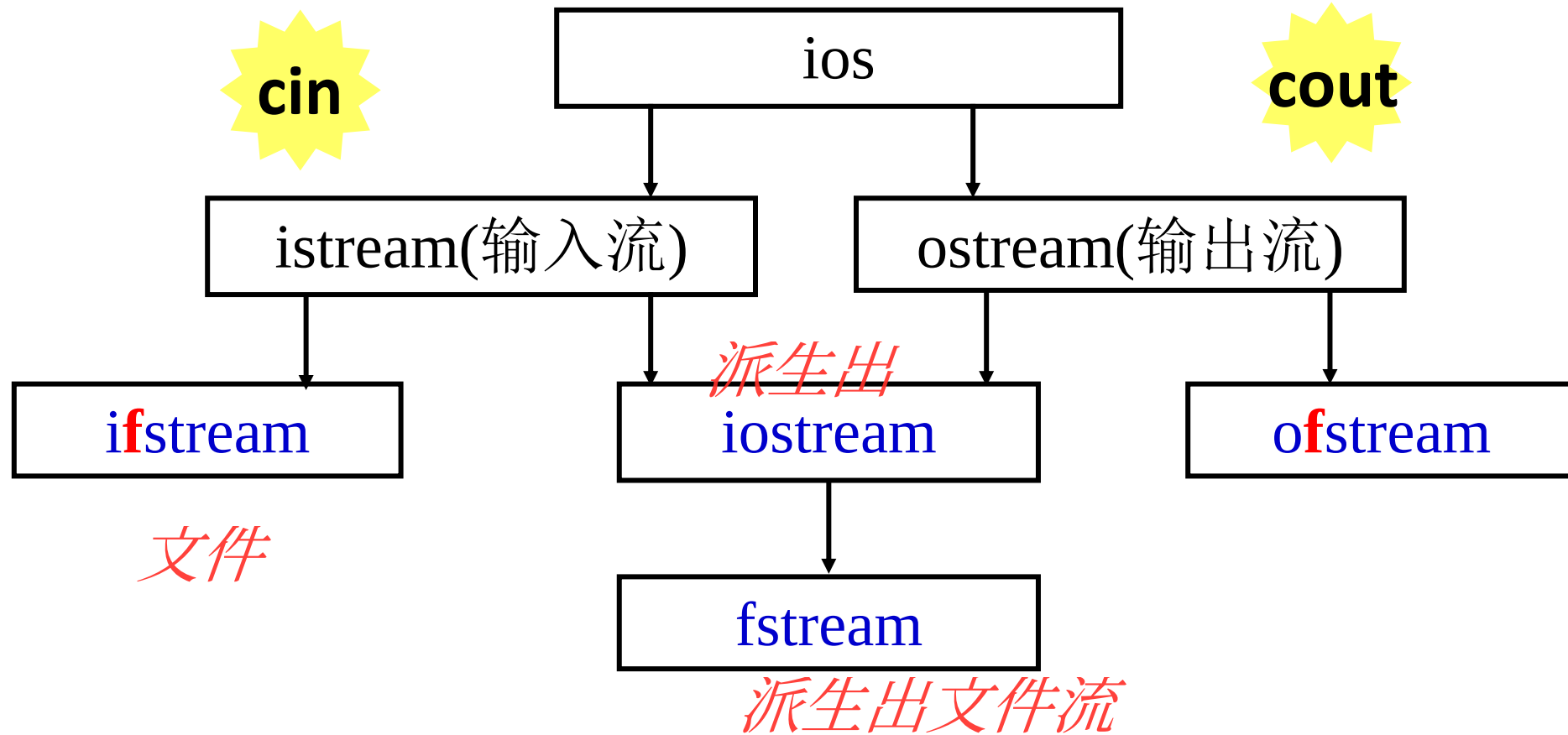


- 标准IO
- 文件IO
- 字符串IO



# 一、C++输入输出

在"iostream.h"中声明



# 一、C++输入输出

| 对象   | 含义    | 对应设备 |
|------|-------|------|
| cin  | 标准输入流 | 键盘   |
| cout | 标准输出流 | 屏幕   |
| cerr | 标准输出流 | 屏幕   |
| clog | 标准输出流 | 屏幕   |

- cerr 和 clog的用法和 cout 完全一样
- cerr 常用来输出警告和错误信息给程序的使用者
- clog 常用来输出程序执行过程中的日志信息（只需开发者看到，无需对普通用户公开）



# 一、C++输入输出

```
#include <iostream>
#include <string>
using namespace std;
void main() {
    string url;
    cin >> url;
    cout << "cout:  " << url<<endl;
    cerr << "cerr:  " << url<<endl;
    clog << "clog:  " << url<<endl;

}
```



# 一、C++输入输出

- 插入运算符<<
- 提取运算符>>

cin>>a;

cout<<a;

对象不能直接输入输出

```
#include <iostream.h>
```

```
class A
```

```
{ float x, y;
```

```
public:
```

```
    A(float a=0, float b=0){ x=a; y=b; }
```

```
    void set(float a, float b){ x=a; y=b; }
```

```
    void show(){ cout<<x<<"\t"<<y<<endl; }
```

```
};
```

```
void main()
```

```
{
```

```
    A a(2,3);
```

```
    a.set(20, 30);
```

```
    a.show( );
```

```
}
```

输入对象数据

输出对象数据

# 一、C++输入输出

在C++中允许用户重载运算符" << "和" >> "，

实现对象输入和输出。重载时，采用友元函数方式。

重载提取运算符一般格式为：

```
friend istream & operator >>(istream &, ClassName &);
```

`cin>>a;`            `operator>>(cin, a)`





# 一、C++输入输出

```
class A
```

```
{ float x, y;
```

```
public:
```

```
.....
```

```
friend istream & operator >>(istream &, A &);
```

```
};
```

```
.....
```

```
A a;
```

```
cin>>a;
```

```
....
```

```
istream & operator >>(istream &is, A &a)
```

```
{ is>>a.x>>a.y;
```

重新定义输入流

```
return is;
```

返回输入流

```
}
```



# 一、C++输入输出

```
#include "iostream.h"
class A
{
    int x,y;
public:
    A(int a=0,int b=0)
        { x=a; y=b; }
    void show()
        { cout<<"x="<<x<<"\t"<<"y="<<y<<endl;}
    friend istream & operator>>(istream &,A &);
};

istream & operator>>(istream &is, A &cc)
{
    is>>cc.x>>cc.y;
    return is;
}

void main()
{
    A a;
    a.show ();
    cin>>a;
    a.show ();
}
```



# 一、C++输入输出

重载插入运算符一般格式为：

```
friend ostream & operator <<(ostream &, ClassName &);
```

cout<<a;        **operator**<<(cout, a)



# 一、C++输入输出

```
class A
```

```
{ float x, y;
```

```
public:
```

```
.....
```

```
friend ostream & operator<<(ostream &, A &);
```

```
};
```

```
.....
```

```
A a;
```

```
cout<<a;
```

```
.....
```

```
ostream & operator <<(ostream &os, A &a)
```

```
{ os<<a.x<<"\t"<<a.y<<endl;
```

重新定义输出流

```
return os;
```

返回输出流

```
}
```



# 一、C++输入输出

```
#include "iostream.h"
class A
{
    int x,y;
public:
    A(int a=0,int b=0)
    {
        x=a;    y=b;    }
    void show()
    {cout<<"x="<<x<<"\t"<<"y="<<y<<endl;    }
    friend istream & operator>>(istream &,A &);
    friend ostream & operator<<(ostream &,A &);
};

istream & operator>>(istream &is,A &cc)
{
    is>>cc.x>>cc.y;    return is;}
ostream & operator<<(ostream &os,A &cc)
{
    os<<"x="<<cc.x<<"\t"<<"y="<<cc.y<<endl;
    return os;}

void main()
{
    A a;
    cout<<a;

    cin>>a;

    cout<<a;    }
```

```
#include <iostream>    //cin、 cout
#include <string>        //string
#include <stdio.h>      //freopen
using namespace std;
void main()
{   string name, url;
    //将标准输入流重定向到 in.txt 文件
    freopen("in.txt", "r", stdin);
    cin >> name >> url;
    //将标准输出重定向到 out.txt文件
    freopen("out.txt", "w", stdout);
    cout << name << "\n" << url;  }
```

```
#include <iostream>

#include <string>

using namespace std;

void main()

{

    string name, url;

    cin >> name >> url;

    cout << name << '\n' << url;

}
```

命令行，分别输入：

- demo.exe <in.txt >out.txt
- demo.exe <in.txt
- demo.exe >out.txt

## 二、标准输出流

利用控制符控制输出格式

`iomanip.h`中定义

| 控制符                          | 描述                |
|------------------------------|-------------------|
| <code>dec</code>             | 数值数据采用十进制表示       |
| <code>hex</code>             | 数值数据采用十六进制表示      |
| <code>oct</code>             | 数值数据采用八进制表示       |
| <code>endl</code>            | 插入换行符，并刷新流        |
| <code>setprecision(n)</code> | 设显示小数精度为n位(包含小数点) |
| <code>setw(n)</code>         | 设域宽为n个字符          |
| <code>setfill(c)</code>      | 设填充字符为c           |





## 二、标准输

```
#include <iostream>
#include <iomanip>
using namespace std;
void main()
{
    int a;
    cout<<"input a:";
    cin>>a;
    cout<<"dec:"<<dec<<a<<endl;
    cout<<"hex:"<<hex<<a<<endl;
    cout<<"oct:"<<setbase(8)<<a<<endl;
    char *pt="China";
    cout<<setw(10)<<pt<<endl;
    cout<<setfill('*')<<setw(10)<<pt<<endl;
    double pi=22.0/7.0;
    cout<<setiosflags(ios::scientific)<<setprecision(8);
    cout<<"pi="<<pi<<endl;
    cout<<"pi="<<setprecision(4)<<pi<<endl;
    cout<<"pi="<<setiosflags(ios::fixed)<<pi<<endl;
```



## 二、标准输出流

```
#include <iostream>
using namespace std;
void main()
{
    int a=21;
    cout.setf(ios::showbase);
    cout<<"dec:"<<a<<endl;          cout.unsetf(ios::dec);
    cout.setf(ios::hex);
    cout<<"hex:"<<a<<endl;          cout.unsetf(ios::hex);
    cout.setf(ios::oct);    cout<<"oct:"<<a<<endl;
    char *pt="China";
    cout.width(10);
    cout<<pt<<endl;
    cout.width(10);
    cout.fill('*');
    cout<<pt<<endl;
    double pi=22.0/7.0;
    cout.setf(ios::scientific);
    cout<<"pi=";
    cout.width(14);
    cout<<pi<<endl;    cout.unsetf(ios::scientific);
    cout.setf(ios::fixed);
    cout.width(12);
    cout.setf(ios::showpos);
    cout.setf(ios::internal);
    cout.precision(6);
    cout<<pi<<endl; }
```



## 二、标准输出流

### cout 常用成员函数

| 成员函数名   | 功能        |
|---------|-----------|
| put()   | 输出单个字符。   |
| write() | 输出指定的字符串。 |



## 二、标准输出流

```
#include <iostream>
#include <string>
using namespace std;
void main()
{
    string str = "nc.ude.uac.www";
    for (int i = str.length() - 1; i >= 0; i--)
    {
        cout.put(str[i]); //从最后一个字符开始输出
    }
    cout.put('\n');
}
```



## 二、标准输出流

```
#include <iostream>
using namespace std;
void main() {
    const char * str = "www.cau.edu.cn";
    cout.write(str, 5);
}
```



## 二、标准输出流

```
#include <iostream>
using namespace std;
void main()
{
    cout.write("www.", 6).write("cau.edu",10).write("cn",4);
}
```



## 二、标准输出流

```
#include <iostream>

using namespace std;

void main()
{
    char *a="BASIC";

    for(int i=4;i>=0;i--)
        cout.put(*(a+i));

    cout.put('\n');
}
```



## 三、标准输入流

### cin 常用成员函数

| 成员函数名                     | 功能                                                                                            |
|---------------------------|-----------------------------------------------------------------------------------------------|
| <b>getline</b> (str,n,ch) | 从输入流中接收 n-1 个字符给 str，当遇到 ch停止读取，默认ch 为 '\0'。                                                  |
| <b>get</b> ()             | 从输入流中读取一个字符，同时该字符会从输入流中消失。                                                                    |
| <b>gcount</b> ()          | 返回上次从输入流提取出的字符个数，常和 get()、getline()、ignore()、peek()、read()、readsome()、putback() 和 unget() 联用。 |
| <b>ignore</b> (n,ch)      | 从输入流中逐个提取字符，但提取出的字符被忽略，不被使用，直至提取出 n 个字符，或者读取字符为 ch。                                           |





### 三、标准输入流

```
#include <iostream>
using namespace std;
void main()
{   char url[30] = {0};
    //读取一行字符串
    cin.getline(url, 10);
    //输出上一条语句读取字符串的个数
    int len=cin.gcount();
    cout << "读取了 " << len << " 个字符" << endl;
    //输出 url 数组存储的字符串
    cout.write(url, len-1);
}
```



### 三、标准输入流

```
#include <iostream>

using namespace std;

void main()
{
    char ch[20];

    cout<<"enter a sentence:"<<endl;

    cin.get(ch,10,'\n');

    cout<<ch<<endl;

}
```



### 三、标准输入流

```
#include <iostream>

using namespace std;

void main()
{
    int n;

    cin.ignore(4);

    cin >> n;

    cout << n;

}
```

输入:

Tel:135

跳过了输入中的前 4 个字符，其余内容被当作整数输入 n 中



## 四、对数据文件的操作与文件流

- 1) 兼容C的文件操作
- 2) C++文件流操作



## 四、对数据文件的操作与文件流

例：从键盘输入字符串，并存到磁盘文件中，回车结束。

```
#include <iostream.h>
#include <stdio.h>
#include <string.h>
void main( )
{
    char str[100];
    cin>>str;

    FILE *fp = fopen("a.txt", "w");
    if(fp==NULL)
    {
        cout<<"can't open the file\n";
        return;
    }
    else
    {
        fwrite(str,sizeof(char),strlen(str),fp) ;
        fclose(fp);
    }
}
```



## 四、对数据文件的操作与文件流

### 文件指针变量定义

```
FILE *fp;
```

**fp是一个指向FILE类型结构体的指针变量**

**通过该指针访问该文件**

**FILE \*指针变量标识符;**



## 四、对数据文件的操作与文件流

### FILE类型

- 为结构体类型的变量，用来存放与文件操作有关信息
- 定义在**stdio.h**中

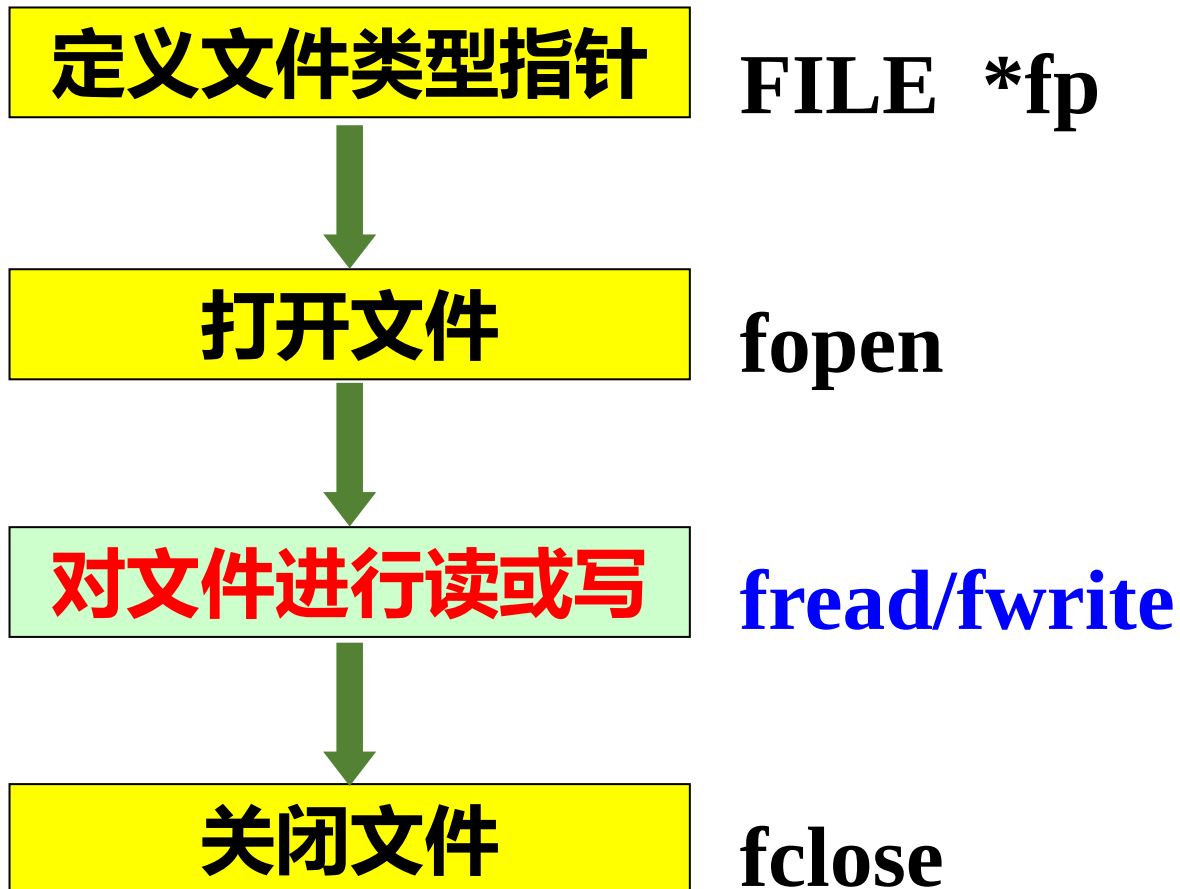
**typedef struct**

```
{short    level;           /* 缓冲区满/空程度 */
 unsigned flags;          /* 文件状态标志 */
 char fd;                 /* 文件描述符 */
 unsigned char hold;      /* 若无缓冲区不读取字符 */
 short bsize;             /* 缓冲区大小 */
 unsigned char *buffer;   /* 数据传送缓冲区位置 */
 unsigned char *curp;     /* 当前读写位置 */
 unsigned istemp;         /* 临时文件指示 */
 short token;             /* 用作无效检测 */
} FILE ;                 /* 结构体类型名 FILE */
```



## 四、对数据文件的操作与文件流

文件操作步骤





## 四、对数据文件的操作与文件流

**打开文件** 将文件从磁盘装入计算机内存,建立文件的各种有关信息,并使文件指针指向该文件,即建立**文件指针**与**文件名**间的**关联**

**fp=fopen(文件名,使用文件方式);**

返回值: FILE类型指针----成功

NULL ----失败

例如: FILE \*fp;

fp=fopen("A1", "r");

含义: 打开名字为A1的文件, 使用文件的方式为“读入”

**fp指向A1文件**



## 四、对数据文件的操作与文件流

### 文件操作方式

“r” (只读) 从**文本**文件中读入数据到程序中

“w” (只写) 将程序中数据写入到**文本**文件中

“a” (追加) 向**文本**文件**尾**增加数据

“rb” (只读) 从**二进制**文件中读入数据到程序中

“wb” (只写) 将程序中数据写入到**二进制**文件中

“ab” (追加) 向**二进制**文件**尾**增加数据

**r**: read    **w**: write    **a**: append    **b**:binary



## 四、对数据文件的操作与文件流

**文件类型** (1) 按文件的内容：(从程序设计的角度看)  
源程序、目标文件、可执行文件、数据文件

(2) 按数据的编码方式（存储形式）：  
文本文件、二进制文件

1、**文本文件**：用ASCII码存储（即一个字符占一个字节）

2、**二进制文件**：以数据在内存中的形式原样存于磁盘



## 四、对数据文件的操作与文件流

例如：

int型数

1268

二进制文件

2 字节

字符文件

4 字节

文本文件形式

00110001

00110010

00110110

00111000

\1'

\2'

\6'

\8'

二进制文件形式

00000100

11110100

其二进制文件存储方式占 2 个字节；

而字符文件存储方式所占字节数 = 该数的数字个数



## 四、对数据文件的操作与文件流

例：从键盘输入字符串，并存到磁盘文件中，回车结束。

```
#include <iostream.h>
#include <stdio.h>
#include <string.h>
void main( )
{
    char str[100];
    cin>>str;

    FILE *fp = fopen("a.txt", "w");
    if(fp==NULL)
    {
        cout<<"can't open the file\n";
        return;
    }
    else
    {
        fwrite(str,sizeof(char),strlen(str),fp);
        fclose(fp);
    }
}
```



## 四、对数据文件的操作与文件流

```
#include <iostream.h>
#include <stdio.h>
void main( )
{
    const char * filename="a.txt";
    FILE *fp=fopen(filename,"r+");
    if(fp==NULL)
    {
        cout<<"cann't open the file!\n";
        return ; }
    char buf[200];
    int n=fread(buf,1,128,fp);
    buf[n]='\0';
    cout<<"the data in file is following:\n"<<buf<<endl;
    for(int i=0;i<n;i++)
    {
        if(buf[i]>='a' && buf[i]<='z')
            buf[i]=buf[i]-32;
        else
            if(buf[i]>='A' && buf[i]<='Z')
                buf[i]=buf[i]+32;
    }
    cout<<"the changed data is : "<<buf<<endl;
    fwrite(buf,1,n,fp);
    fclose(fp); }
```



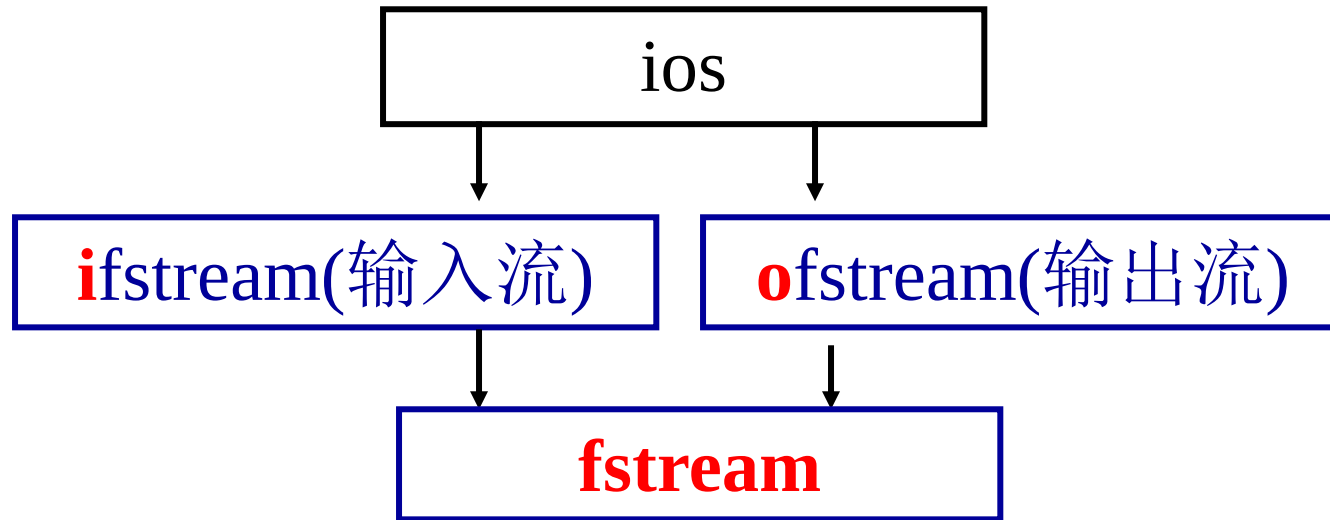
## 四、对数据

```
#include <iostream.h>
#include <stdio.h>
void main( )
{
    const char * filename="a.txt";
    FILE *fp=fopen(filename,"r+");
    if(fp==NULL)
    {
        cout<<"cann't open the file!\n";
        return ; }
    char buf[200];
    int n=fread(buf,1,128,fp);
    buf[n]='\0';
    cout<<"the data in file is following:\n"<<buf<<endl;
    for(int i=0;i<n;i++)
    {
        if(buf[i]>='a' && buf[i]<='z')
            buf[i]=buf[i]-32;
        else
            if(buf[i]>='A' && buf[i]<='Z')
                buf[i]=buf[i]+32;
    }
    cout<<"the changed data is : "<<buf<<endl;
    rewind(fp);
    fwrite(buf,1,n,fp);
    fclose(fp); }
```



## 四、对数据文件的操作与文件流

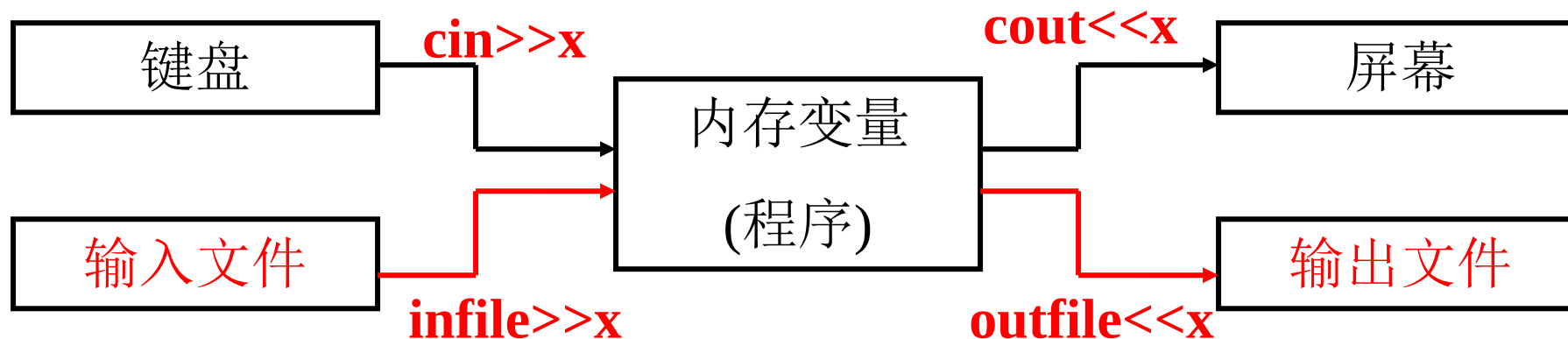
C++在头文件fstream.h中定义了C++的文件流类体系  
其中定义了各种文件操作运算符及函数





## 四、对数据文件的操作与文件流

程序对文本文件的操作与对键盘、显示器的操作比较：



进行**文本文件**操作时，将输入文件看成键盘，将输出文件看成显示器，格式不变。

只需在程序中增加打开与关闭文件的语句。



## 四、对数据文件的操作与文件流

### 文本文件的打开与关闭

在程序内定义一个文件类**对象**，由**该对象**与文件发生联系，程序内所有的与文件的操作都是对**该对象**操作。

```
fstream  infile , outfile;
```

两个对象，可以联系两个输入输出文件

```
ifstream  infile;
```

对象只能联系输入文件

```
ofstream  outfile;
```

对象只能联系输出文件



## 四、对数据文件的操作与文件流

### 用对象打开文件

```
ifstream infile; //定义输入文件类对象
```

```
infile.open("myfile1.txt");//利用函数打开某一文件
```

打开文件的作用：建立文件流对象与要使用的文件名间联系

用infile代表文件“myfile1.txt”

用完文件后,使用成员函数关闭文件

```
infile.close();
```



| 模式标记                               | 作用                                                                    |
|------------------------------------|-----------------------------------------------------------------------|
| ios::in                            | 读取数据。如果文件不存在，则打开出错。                                                   |
| ios::out                           | 写入数据。如果文件不存在，则新建该文件；如果文件原来就存在，则打开时清除原来的内容。                            |
| ios::app                           | 打开文件，用于在其尾部添加数据。如果文件不存在，则新建该文件                                        |
| ios::ate                           | 打开一个 <b>已有</b> 的文件，并将文件读指针指向文件末尾。<br>如果文件不存在，则 <b>打开出错</b>            |
| ios:: trunc                        | 打开文件时会 <b>清空</b> 内部存储的所有数据，单独使用时与 ios::out 相同                         |
| ios::binary                        | 以 <b>二进制</b> 方式打开文件。若不指定此模式，则以文本模式打开。                                 |
| ios::in   ios::out                 | 打开已存在的文件，既可读取其内容，也可向其写入数据。<br>文件刚打开时，原有内容保持不变。如果文件不存在，则 <b>打开出错</b> 。 |
| ios::in   ios::out  <br>ios::trunc | 打开文件，既可读取其内容，也可向其写入数据。<br>如果文件本来就存在，则打开时清除原来的内容；<br>如果文件不存在，则新建该文件。   |

**ios::binary** 可以和其他模式标记组合使用，例如：

**ios::in | ios::binary**表示用二进制模式，以读取的方式打开文件。

**ios::out | ios::binary**表示用二进制模式，以写入的方式打开文件。

## 四、对数据文件的操作与文件流

```
#include "iostream.h"
#include "fstream.h"

void main()
{
    char filename1[256],filename2[256];
    char buf[300];
    cout<<"Input source file name: ";    cin>>filename1;
    cout<<"Input destination: ";        cin>>filename2;
    fstream infile,outfile;
    infile.open(filename1,ios::in);
    outfile.open(filename2,ios::out);
    while(infile.getline(buf,300))
        outfile<<buf<<endl;
    outfile.close();
    infile.close();
}
```



## 四、对数据文件的操作与文件流

char ch, str[300];

ifstream infile("myfile1.txt");

ofstream outfile("myfiel2.txt");

从键盘输入一个字符: cin.get(ch);

从文件输入一个字符: infile.get(ch);

向显示器输出一个字符: cout.put(ch);

向文件输出一个字符: outfile.put(ch);

从键盘输入一行字符: cin.getline(str,300);

从文件输入一行字符: infile.getline(ch,300);

从文件输入一字符或一行字符，当输入至文件尾时，函数返回值为0，可以据此来判断循环结束。



## 四、对数据文件的操作与文件流

### 二进制文件的读写操作

若在文件的打开方式中没有特别说明，打开的文件均为ASCII码文件

若要打开二进制文件，则要特别说明并用特定的读写函数

```
fstream  infile,outfile;
```

```
infile.open(“inf1.dat”, ios::in| ios::binary);
```

```
outfile.open(“outf1.dat”, ios::out| ios::binary);
```





## 四、对数据文件的操作与文件流

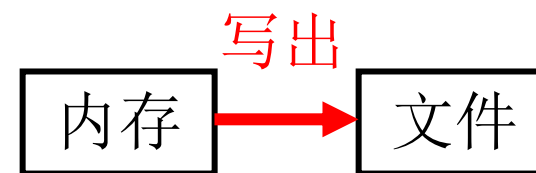
输出函数：

一次输出的字节数

```
outfile.write( char *, int )
```

输出文件对象名

要输出的数据在内存中的地址



地址要强制转换成字符型

```
int i=4;
```

```
outfile.write((char *)&i, sizeof(int)); //向文件输出一个整型数i
```

```
int a[10]={0,1,2,3,4,5,6,7,8,9};
```

```
outfile.write((char *)a, 10*sizeof(int)); //向文件输出一个整型数组a
```



## 四、对数据文件的操作与文件流

```
#include <fstream>
#include <iostream>
using namespace std;
struct student
{
    char name[20];
    int num;
    int age;
    char sex;
};
void main()
{
    student stud[3]={"Li",1001,18,'f',"Fun",1002,19,'m',"Wang",1004,17,'f'};
    ofstream outfile("stud.dat",ios::binary);
    if(!outfile)
    {
        cerr<<"open error!"<<endl;
        abort(); }
    for(int i=0;i<3;i++)
        outfile.write((char *)&stud[i],sizeof(stud[i]));
    outfile.close();
}
```



## 四、对数据文件的操作与文件流

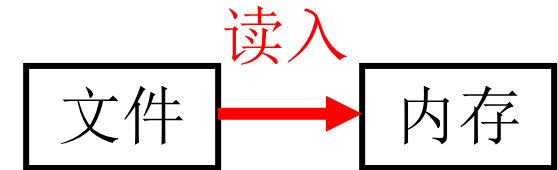
输入函数：

一次读入的字节数

```
infile.read( char *, int )
```

输入文件对象名

数据进入的内存地址



```
int i;
```

地址要强制转换成字符型

```
infile.read((char *)&i, sizeof(int)); // 从文件中输入一个整型数到i
```

```
int a[10];
```

```
infile.read((char *)a, 10*sizeof(int)); // 从文件中输入十个整型数到a
```



## 四、对数据文件的操作与文件流

```
#include <fstream>
#include <iostream>
using namespace std;
struct student
{
    char name[20];    int num; int age;  char sex; };
void main()
{
    student stud[3];
    int i;
    ifstream infile("stud.dat",ios::binary);
    if(!infile)
    {
        cerr<<"open error!"<<endl;
        abort();    }
    for(i=0;i<3;i++)
        infile.read((char*)&stud[i],sizeof(stud[i]));
    infile.close();
    for(i=0;i<3;i++)
    {
        cout<<"NO."<<i+1<<endl;
        cout<<"name:"<<stud[i].name<<endl;
        cout<<"num:"<<stud[i].num<<endl;;
        cout<<"age:"<<stud[i].age<<endl;
        cout<<"sex:"<<stud[i].sex<<endl<<endl;    }
}
```



## 四、对数据文件的操作与文件流

### 判断二进制文件是否读到文件尾？

`infile.eof( )`

当到达文件结束位置时，该函数返回一个非零值； 否则返回零。

```
fstream infile;
```

```
infile.open("data1.dat",ios::in|ios::binary);
```

```
if(!infile) 判断打开是否出错
```

```
{ cout<<"Open Error!\n"; exit(1); }
```

```
char str[300];
```

```
while(!infile.eof()) 判断是否读到文件尾
```

```
infile.read(str, 300);
```

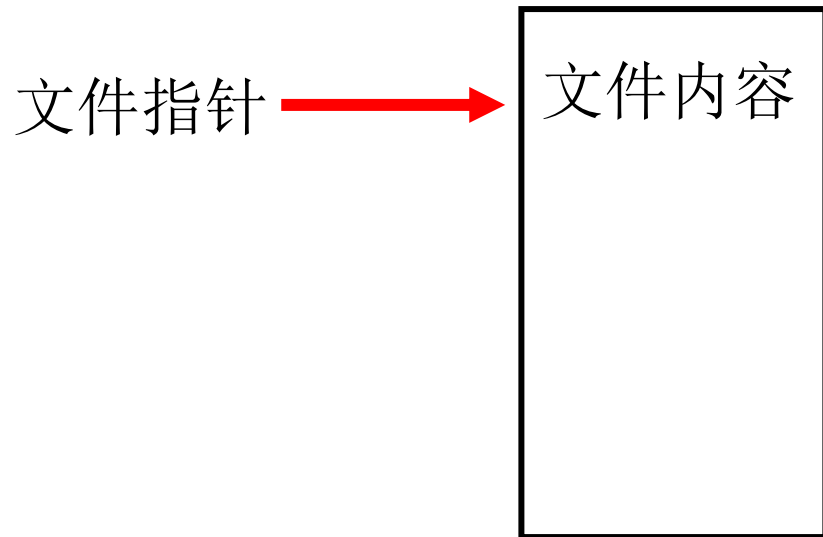


## 四、对数据文件的操作与文件流

### 文件指针

当一打开文件，文件指针位于文件头，并随着读写字节数的多少顺序移动

可以利用成员函数随机移动文件指针



## 四、对数据文件的操作与文件流

| 函数名                     | 作用               |
|-------------------------|------------------|
| <b>gcount()</b>         | 最后一次输入所读入的字节数    |
| <b>tellg()</b>          | 输入文件位置标记的当前位置    |
| <b>tellp()</b>          | 输出文件位置标记的当前位置    |
| <b>seekg(文件中的位置)</b>    | 将输入文件位置标记移到指定的位置 |
| <b>seekg(位移量, 参考位置)</b> | 以参考位置为基础移动若干字节   |
| <b>seekp(文件中的位置)</b>    | 将输出文件位置标记移到指定的位置 |
| <b>seekp(位移量, 参考位置)</b> | 以参考位置为基础移动若干字节   |



## 四、对数据文件的操作与文件流

### 随机读取二进制文件

`infile.seekg(int);`//将文件指针移动到由参数指定的字节处

`infile.seekg(int, ios::_dir);`

移动的字节数

相对位置

`_dir:` `beg:` 文件头  
`cur:` 当前位置  
`end:` 文件尾

`infile.seekg(100, ios::beg);`//移动到距文件头100个字节

`infile.seekg(-100, ios::cur);`//移动到距当前位置前100个字节

`infile.seekg(-500, ios::end);`//移动到距文件尾前500个字节





# 四、对数据文件的操作与文件流

```
#include <fstream>
#include <iostream>
#include <string.h>
using namespace std;
struct student
{
    int num;        char name[20];        float score;};
void main()
{
    int i;
    student stud[5]={ 1001,"Li",85,1002,"Fun",97.5,1004,"Wang",54,1006,"Tan",76.5,1010,"ling",96};
    fstream iofile("stud.dat",ios::in|ios::out|ios::binary);
    if(!iofile)
    {
        cerr<<"open error!"<<endl;
        abort(); }
    for(i=0;i<5;i++)
        iofile.write((char *)&stud[i],sizeof(stud[i]));
    student stud1[5];
    for(i=0;i<5;i=i+2)
    {
        iofile.seekg(i*sizeof(stud[i]),ios::beg);
        iofile.read((char *)&stud1[i/2],sizeof(stud1[i]));
        cout<<stud1[i/2].num<<" "<<stud1[i/2].name<<" "<<stud1[i/2].score<<endl;}
    cout<<endl;
    stud[2].num=1012;
    strcpy(stud[2].name,"Wu");
    stud[2].score=60;
    iofile.seekp(2*sizeof(stud[0]),ios::beg);
    iofile.write((char *)&stud[2],sizeof(stud[2]));
    iofile.seekg(0,ios::beg);
    for(i=0;i<5;i++)
    {
        iofile.read((char *)&stud[i],sizeof(stud[i]));
        cout<<stud[i].num<<" "<<stud[i].name<<" "<<stud[i].score<<endl; }
    iofile.close();}
```



## 四、对数据文件的操作与文件流

### 练习：抽奖程序

- 1、编程生成文本文件data.txt，文件内容是10个手机号码（手机号码前4位为1871，后面随机生成）（**尝试用多种方式实现**）
- 2、编程：从文本文件data.txt中读取抽奖者的个人信息（**尝试用多种方式实现**）
- 3、预设一、二、三等奖的人数（一等奖 1 名、二等奖 2 名、三等奖 5名）
- 4、按下任意键开始抽奖，并把中奖人的手机号显示在屏幕上，反复抽出一、二、三等奖。如果抽完就不能再抽。

分析：

为保证抽奖的公正性，要使用随机函数生成获奖者。

抽奖者不能重复获奖。



## 四、对数据文件的操作与文件流

编程生成文件**data.txt**，文件内容是**10个手机号码**  
(手机号码前**4位**为**1871**，后面随机生成)



## 四、对数据文件的操作与文件流

```
#include "iostream.h"
#include "stdio.h"
#include "stdlib.h"
#include "time.h"
#include "fstream.h"
void main()
{
    int i;    char ch;  FILE *fp;
    fp=fopen("test.txt","wb");
    if(fp==NULL)
    {
        cout<<"cann't open file"<<endl;
        return;    }
    srand((unsigned)time(NULL));
    for(i=1;i<70;i++)
    {
        if(i%7==0)
        {
            ch='\0';
            fputc(ch,fp);        }
        else
        {
            ch=rand()%('9'-'0'+1)+'0';
            fputc(ch,fp);        }
    }
    fclose(fp);
}
```

```
#include "iostream.h"
#include "stdio.h"
#include "stdlib.h"
#include "time.h"
#include "fstream.h"
void main()
{
    int i;    char nm[120];    char ch;
    ofstream outfile("number.txt");
    if(!outfile)
    {
        cerr<<"open error!"<<endl;
        abort();    }
    srand((unsigned)time(NULL));
    for(i=1;i<120;i++)
    {
        if(i%12==0)
        {
            ch='\0';
            nm[i]=ch;
            outfile<<nm[i];    }
        else
        {
            ch=rand()%('9'-'0'+1)+'0';
            nm[i]=ch;
            outfile<<nm[i];    }
    }
    outfile.close();}
```

## 五、字符串流

<http://www.cplusplus.com/info/>

C++ 为处理字符串提供两种类型：**C 字符串**和 **string 类**

C 字符串是存储在连续内存位置中的字符序列，并以 null 字符结尾

null 字符是 ASCII 码为 0 的字符

程序中，null 字符通常写成 '\0'



## 五、字符串流

```
#include <iostream>
#include <sstream>
using namespace std;
void main()
{
    char carray[20]="helloChina";
    char *cp;
    cp=carray;
    for(int i=0;i<20;i++)
    {
        if(i<strlen(carray))
            //cout<<carray[i];
            cout<<*(cp++);
        else
            break;
    }
}
```



## 五、字符串流

```
#include <iostream>
#include <sstream>
using namespace std;
void main()
{
    char carray[20]="helloChina";
    char *cp;
    cp=carray;
    cout<<cp<<endl;
    // cout<<carray<<endl;
}
```



## 五、字符串流

|               |                                                                                                                                                  |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>strlen</b> | 返回 C 字符串的长度（不包括 null 终止符）<br>例：len = strlen(name);                                                                                               |
| <b>strcat</b> | 将第二个字符串的内容附加到第一个 C 字符串后面(第一个字符串被改变，第二个字符串保持不变)。<br>例：strcat(string1, string2);                                                                   |
| <b>strcpy</b> | 将第二个 C 字符串复制到第一个 C 字符串。第二个 C 字符串保持不变<br>例：strcpy(string1, string2);                                                                              |
| <b>strcmp</b> | 如果 string1 和 string2 相同，则该函数返回 0；<br>如果 string2 按字母顺序大于 string1，则返回负数；<br>如果 string2 按字母顺序小于 string1，则返回一个正数。<br>例：if (strcmp(string1, string2)) |





## 五、字符串流

string 类可用 "=" 赋值，用 "==" 等值比较，用 "+" 做串联。

包含头文件 `<string>`

string 类支持 cin 方式和 cout 方式输入输出方式。

```
#include <iostream>
#include <string>
using namespace std;
void main()
{
    string stuff;
    cin >> stuff;
    cout<<stuff;
}
```



## 五、字符串流

### string 类构造函数

|                         |                    |
|-------------------------|--------------------|
| string()                | 创建空字符串             |
| string(const char *s)   | 用 C 字符串创建string 对象 |
| string(const string &s) | 用现有字符串创建新的字符串      |

```
#include <iostream>
#include <string>
using namespace std;
void main()
{
    string stuff("hello");
    string stuff1=stuff;
    cout<<stuff1;
}
```



# 五、字符串流

## string 重载运算符

|       |
|-------|
| >>    |
| <<    |
| =     |
| +=    |
| +     |
| []    |
| 关系运算符 |

```
#include <iostream>
#include <string>
using namespace std;
void main()
{   string s1("hello");
    string s2="beijing";
    if(s1==s2)
        cout<<s1[1];
    else
        cout<<s2+"china";
}
```



## 五、字符串流

|                             |                              |
|-----------------------------|------------------------------|
| <b>s.append(str);</b>       | 将 str 追加到 s                  |
| <b>s.append(str, p, n);</b> | str 中从位置 p 开始的 n 个字符被附加到 s 后 |
| <b>s.append(str, n);</b>    | C 字符串 str 的前 n 个字符被附加到 s 后   |
| <b>s.append(n, ch);</b>     | 将字符 ch 的 n 个副本附加到 s 后        |
| <b>s.assign(str);</b>       | 将 str 赋值给 s                  |
| <b>s.assign(str, p, n);</b> | 从位置 p 开始，str 中的 n 个字符被赋值给 s  |
| <b>s.at(p);</b>             | 返回字符串中 p 位置的字符               |
| <b>s.begin();</b>           | 返回 s 中第一个字符的指针               |
| <b>s.capacity();</b>        | 返回 s 存储的大小                   |
| <b>s.clear();</b>           | 删除 s 中存储的所有字符                |
| <b>s.compare(str);</b>      | 比较 s 和 str                   |



## 五、字符串流

|                          |                                           |
|--------------------------|-------------------------------------------|
| <b>s.empty();</b>        | 如果 s 为空则返回 <b>true</b> ，否则返回 <b>false</b> |
| <b>s.end();</b>          | 返回指向 s 中最后一个字符的指针                         |
| <b>s.erase(p, n);</b>    | 从 s 的位置 p 开始，删除 n 个字符                     |
| <b>s.find(str);</b>      | 返回在 s 中找到的 str 字符串的第一个位置                  |
| <b>s.find(ch);</b>       | 返回在 s 中找到的 ch 字符的第一个位置                    |
| <b>s.insert(p, str);</b> | 将 str 的副本插入到 s 中，从位置 p 开始                 |
| <b>s.length();</b>       | 返回 s 中字符串的长度                              |
| <b>s.size();</b>         | 返回 s 中字符串的长度                              |
| <b>s.swap(str);</b>      | 交换 s 和 str 的内容                            |



## 五、字符串流

```
#include <iostream>
#include <string>
using namespace std;
void main()
{ string s1("aaaaaaaa");
    string s2="beijing";
    // s1.append(s2);// s1.append(s2,2,3);// s1.append(2,'A');
    // s1.assign(s2);// s1.assign(s2,2,3);
    // cout<<s2.at(2)<<endl;
    // cout<<s2.begin()<<endl;
    // cout<<s2.capacity()<<endl;
    // cout<<s2.size()<<endl;
    // cout<<s1.compare(s2)<<endl;//cout<<s2.compare(s1)<<endl;
    // cout<<s2.empty()<<endl;
    // cout<<*(s2.end()-1)<<endl;
    // cout<<s2.erase(2,3)<<endl;
    // cout<<s2;
    // string s3="eij";// cout<<s2.find(s3)<<endl;
    // cout<<s2.find('b')<<endl;
    // cout<<s1.insert(2,"bb");
    // cout<<s2.substr(2,3)<<endl;
    // s1.swap(s2);
    cout<<s1;
}
```

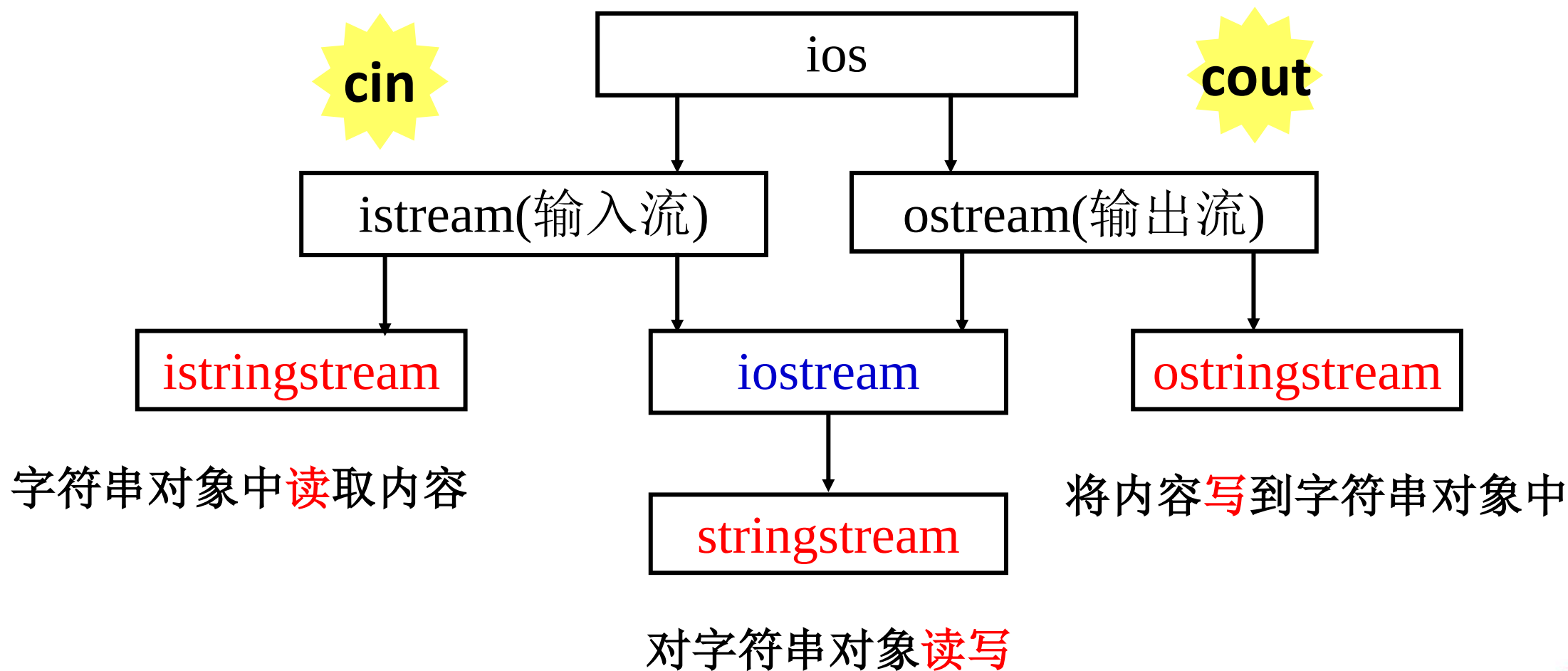


## 五、字符串流

```
#include <iostream>
#include <string>
using namespace std;
void main(){
    string s = "1234567890";
    for(int i=0,len=s.length(); i<len; i++)
    {    cout<<s[i]<<" ";    }
    cout<<endl;
    s[5] = '5';
    cout<<s<<endl;
}
```



## 五、字符串流



包含在头文件 `sstream` 中





## 五、字符串流

在C++有两种字符串流:基本一样。

- **sstream** 基于std::string编写的, 包含:

class istringstream;

class ostreamstream;

class stringbuf;

class stringstream;

class .....

**ostreamstream::str()**

返回std::string类型的字符串

- **strstream** 基于C类型字符串char\*编写的, 包含:

class strstreambuf;

class istrstream;

class ostrstream;

class strstream;

**ostrstream::str()**

返回char\*类型的字符串



## 五、字符串流

空格为字符串的内部分界

```
#include <iostream>
#include <sstream>
using namespace std;
void main()
{
    istringstream istr;
    istr.str("1 56.7");
    // istringstream istr("1 56.7");
    cout << istr.str()<<endl;
    int a;
    float b;
    istr>>a;
    cout<<a<<endl;
    istr>>b;
    cout<<b<<endl;
}
```



## 五、字符串流

```
#include <iostream>
#include <sstream>
using namespace std;
void main()
{
    ostringstream ostr;
    ostr.put('d');
    ostr.put('e');
    ostr<<"fg";
    string gstr = ostr.str();
    cout<<gstr;
}
```

**put()**或者左移操作符向**ostr**插入单个字符或字符串



## 五、字符串流

```
#include <sstream>
#include <iostream>
using namespace std;
void main()
{
    ostringstream ostr;
    int price= 130;
    char* ps = " to C++";
    ostr << "Pay only$" << price << ps << endl;
    string msg = ostr.str();
    cout<<msg<<endl;
}
```

**str()成员函数将stringstream对象转换为string字符串**



## 五、字符串流

**stringstream**通常是用来做数据转换的

```
#include <string>
#include <sstream>
#include <iostream>
using namespace std;
void main()
{
    stringstream stream;
    string result;
    int i = 1000;
    stream << i;
    stream >> result;
    cout << result << endl;
}
```



## 五、字符串流

```
#include <iostream>
#include <sstream>
using namespace std;
void main()
{   stringstream sstr;
    //-----int转string-----
        int a=100;
        string str;
        sstr<<a;
        sstr>>str;
        cout<<str<<endl;
    //-----string转char[]-----
        sstr.clear();
        string name = "beijing";
        char cname[200];
        sstr<<name;
        sstr>>cname;
        cout<<cname;    }
```

多次转换时须

调用stringstream的成员函数clear()



## 五、字符串流

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std;
void main()
{
    string s = "ab,cd,e,fg,h";
    int n = s.size();
    for (int i = 0; i < n; ++i)
    {
        if (s[i] == ',')
        {
            s[i] = ' ';
        }
    }
    stringstream out(s);
    string str;
    // out >> str;    // cout << str << endl;
    while (out >> str)
    {
        cout << str << ' ';
    }
    cout << endl;
}
```







假设A,B,C,D,....., X,Y,Z这26个英文字母

分别等于百分之1,2,.....,24,25,26

尝试计算: **HARD WORK**  $8+1+18+4+23+15+18+11=98\%$

**KNOWLEDGE** 96%

**LOVE** 54%

**LUCK** 47%

请编程计算下列单词

**MONEY (72) STUDY (89) ATTITUDE (100)**

最好做成，提示输入字符串，然后计算字符串的百分比。字符串以回车作为输入结束

请综合输入输出流、文件流和字符串流、特别是文件流中指针相关函数进行文件随机位置读取等相关内容，自主设计一个问题，并编程加以实现。

请提交一个word文档，包括：

- 1、问题的描述
- 2、问题实现代码
- 3、程序运行结果分析

## 五、字符串流

```
#include <sstream>
#include <iostream>
using namespace std;
struct student
{
    int num;
    char name[20];
    float score;
};
void main()
{
    student stud[3]={1001,"Li",78,1002,"Wang",89.5,1004,"Fun",90};
    char c[50];
    ostringstream strout(c,30);
    for(int i=0;i<3;i++)
        strout<<stud[i].num<<stud[i].name<<stud[i].score;
    strout<<ends;
    cout<<"array c:"<<endl<<c<<endl;
}
```



## 五、字符串

```
#include <sstream>
#include <iostream>
using namespace std;
void main()
{
    char c[50]="12 34 65 -23 -32 33 61 99 321 32";
    int a[10],i,j,t;
    cout<<"array c:"<<c<<endl;
    istringstream strin(c,sizeof(c));
    for(i=0;i<10;i++)
        strin>>a[i];
    cout<<"array a:";
    for(i=0;i<10;i++)
        cout<<a[i]<<" ";
    cout<<endl;
    for(i=0;i<9;i++)
        for(j=0;j<9-i;j++)
            if(a[j]>a[j+1])
            {
                t=a[j];a[j]=a[j+1];a[j+1]=t;}
    ostringstream strout(c,sizeof(c));
    for(i=0;i<10;i++)
        strout<<a[i]<<" ";
    strout<<ends;
    cout<<"array c:"<<c<<endl;
}
```



谢 谢

